APNA COLLEGE

AVL Tree Deletion

BST delete is a recursive function in which, after deletion, we get pointers to all ancestors one by one in a bottom up manner. So we don't need a parent pointer to travel up. The recursive code itself travels up and visits all the ancestors of the deleted node.

a. *Perform the normal BST deletion.*
b. *The current node must be one of the ancestors of the deleted node. Update the height of the current node.*
c. *Get the balance factor (left subtree height – right subtree height) of the current node.*
d. *If the balance factor is greater than 1, then the current node is unbalanced and we are either in the Left Left case or Left Right case. To check whether it is Left Left case or Left Right case, get the balance factor of the left subtree. If the balance factor of the left subtree is greater than or equal to 0, then it is Left Left case, else Left Right case.*
e. *If the balance factor is less than -1, then the current node is unbalanced and we are either in the Right Right case or Right Left case. To check whether it is a Right Right case or Right Left case, get the balance factor of the right subtree. If the balance factor of the right subtree is smaller than or equal to 0, then it is Right Right case, else Right Left case.*

```
//for non-empty BST, return the node with MIN data
   public static Node getMinNode(Node root) {
        Node curr = root;
        //MIN data is at left-most node
        while (curr.left != null)
        curr = curr.left;
        return curr;
   }
   public static Node deleteNode(Node root, int key) {
        // perform usual BST delete
        if (root == null) {
            return root;
```